

An Independent Transformer Oracle for Auditing Lean 4’s `native_decide`

LLMCompiler

Abstract

Lean 4’s `native_decide` tactic discharges a decidable proposition by *compiling* its `Decidable` instance to native code, executing it, and trusting the result. This is fast, but it enlarges the trusted computing base to include the Lean→C compiler, every `@[extern]/@[implemented_by]` fast path the procedure touches, and the physical CPU. We present LLMCompiler, a system that compiles ordinary programs into the weights of a transformer and executes them there, and we use it as an *independent execution oracle* to audit `native_decide`. We give an operational semantics for the target ISA (NISA), show how a weight-constructed transformer realises its one-step relation in a bipolar $\{-1, +1\}$ tensor encoding, and describe a freestanding mini Lean runtime that lets the emitted C run through the pipeline. We then formalise the audit as a *sound but incomplete falsifier*: a disagreement between the compiled procedure and the independent evaluators is a proof of a fault, whereas agreement is only evidence. We discuss the threat model (including silent hardware data corruption) and report case studies in which the method found real defects, both in Lean’s ecosystem and, applying the same methodology, in unrelated mature libraries.

1 Introduction

A proof assistant is only as trustworthy as the code that decides its proof obligations. Lean 4 offers two tactics for closing a *decidable* goal P :

- `decide`, which evaluates the `Decidable P` instance *inside the kernel*; the only trusted component is the kernel’s reduction machine;
- `native_decide`, which *compiles* the instance to C, then to native code, runs it, and accepts P if the program returns `isTrue`.

`native_decide` is often orders of magnitude faster, which makes it indispensable for verification tasks that reduce to large finite computations. The cost is epistemic: correctness of the proof now depends on the compiler, on any hand-written fast paths substituted for reference definitions at runtime, and on the hardware. These dependencies are exactly the ones discharged by Lean’s axioms `ofReduceBool` and `trustCompiler`. If any of them fails for the specific computation, `native_decide` can accept a false proposition without any indication.

The contribution of this paper is a practical, *independent* way to check what `native_decide` actually computed. LLMCompiler lowers C (including the C that Lean emits) to a small register machine, NISA, and executes NISA on a transformer whose weights are *constructed* so that one forward pass performs one instruction. Because that executor shares no code with a conventional CPU — it computes in a bipolar $\{-1, +1\}$ encoding as dense tensor algebra — running the same decision procedure on it, and cross-checking against a separate mathematical oracle, gives a differential test whose disagreements localise a genuine fault.

The governing principle. Following Dijkstra, testing can reveal the presence of bugs but not their absence. Re-running a procedure and obtaining the *same* answer proves nothing: the two runs may share a defect. Obtaining a *different* answer is a falsifier: it proves at least one path is wrong (§6). Every design choice below serves to make disagreements meaningful and to maximise the independence of the paths being compared.

2 Background: `native_decide` and its trust surface

Let P be a proposition with an instance $d : \text{Decidable } P$. Reducing d yields either `isTrue h` (with $h : P$) or `isFalse h`. `native_decide` works by evaluating the boolean `decide P` through Lean’s *compiler* rather than its kernel and appealing to

$$\text{ofReduceBool} : (\text{decide } P \rightsquigarrow_{\text{native}} \text{true}) \rightarrow P,$$

where $\rightsquigarrow_{\text{native}}$ denotes evaluation of the compiled program. The trusted surface of a single `native_decide` call is therefore the set

$$\mathcal{T}(P) = \underbrace{\mathcal{C}}_{\text{Lean} \rightarrow \text{C} \rightarrow \text{native}} \cup \underbrace{\mathcal{F}(d)}_{\text{fast paths reached by } d} \cup \underbrace{\mathcal{H}}_{\text{CPU}}.$$

The most commonly implicated component is $\mathcal{F}(d)$: standard-library functions carry `@[extern]` or `@[implemented_by]` annotations that replace the reference Lean definition with hand-written C or a faster Lean routine during compiled execution. A discrepancy between a fast path and its reference definition at an edge case is invisible to `decide` (which runs the reference) but is exactly what `native_decide` runs. Our audit targets $\mathcal{T}(P)$ as a whole by re-executing d on machinery that shares none of $\mathcal{C}$, $\mathcal{F}(d)$, or $\mathcal{H}$.

3 The NISA abstract machine

NISA is a 40-opcode, 32-bit, RISC-V-derived instruction set. We give its small-step semantics.

Definition 1 (Words and stores). Let $\mathbb{W} = \mathbb{Z}/2^{32}\mathbb{Z}$ be the set of machine words, $\text{Reg} = \{0, \dots, 31\}$ the register indices, $\text{Byte} = \mathbb{Z}/2^8\mathbb{Z}$, and $\text{Addr} = \{0, \dots, N-1\}$ a byte-addressed memory of size N . A *register file* is $R : \text{Reg} \rightarrow \mathbb{W}$ and a *memory* is $M : \text{Addr} \rightarrow \text{Byte}$. Multi-byte accesses are little-endian: $\text{wd}_M(a) = \sum_{i=0}^3 M(a+i) 2^{8i} \in \mathbb{W}$.

Definition 2 (Instructions and programs). An instruction is a tuple (op, a, b, c) with op one of the 40 opcodes and a, b, c operands (register indices or immediates, per opcode). A program is a finite sequence $P = \langle I_0, \dots, I_{n-1} \rangle$. A *configuration* is $\sigma = (pc, R, M)$ with $pc \in \mathbb{N}$.

Definition 3 (One-step relation). The transition $\sigma \rightarrow_P \sigma'$ is defined by cases on $I_{pc} = (op, a, b, c)$. Writing $R' = R[a \mapsto v]$ for the register update and $pc' = pc+1$ unless stated otherwise, representative rules are:

$$\begin{array}{ll} \text{ADD} & v = R(b) + R(c) \bmod 2^{32}, \quad (pc, R, M) \rightarrow (pc', R', M) \\ \text{MUL} & v = R(b) \cdot R(c) \bmod 2^{32}, \quad (pc, R, M) \rightarrow (pc', R', M) \\ \text{SLTU} & v = [R(b) < R(c)], \quad (pc, R, M) \rightarrow (pc', R', M) \\ \text{LOAD} & v = \text{wd}_M(R(b)+c), \quad (pc, R, M) \rightarrow (pc', R', M) \\ \text{STORE} & M' = M[R(b)+c \mapsto R(a)], \quad (pc, R, M) \rightarrow (pc, R, M') \\ \text{BNE} & pc' = \begin{cases} c & R(a) \neq R(b) \\ pc+1 & \text{else} \end{cases}, \quad (pc, R, M) \rightarrow (pc', R, M) \\ \text{HALT} & \text{no successor (terminal).} \end{array}$$

Signed opcodes (SLT, BLT, DIV, SRA, ...) interpret operands via the two’s-complement bijection $\iota : \mathbb{W} \rightarrow \{-2^{31}, \dots, 2^{31}-1\}$. The remaining opcodes (bitwise, shifts, immediates, byte/half memory, high-multiply MULH*, jumps) are analogous.

Definition 4 (Whole-program evaluation). Let $\longrightarrow_P^*$ be the reflexive-transitive closure of $\longrightarrow_P$. For an initial configuration $\sigma_0 = (0, R_0, M_0)$, if $\sigma_0 \longrightarrow_P^* (pc, R, M)$ with $I_{pc} = \text{HALT}$, the machine *halts* and its result is $\llbracket P \rrbracket(\sigma_0) = R(10)$ (the ABI return register `a0`). If no such terminal configuration is reachable within a cycle budget B , evaluation is $\perp$ (timeout).

4 The bipolar transformer executor

We now realise $\longrightarrow_P$ as a single transformer layer acting on a tensor-encoded configuration. The executor is not trained; its weights are *constructed* to compute Definition 3 exactly, in arithmetic disjoint from a conventional ALU.

Definition 5 (Bipolar encoding). Let $\beta : \{0, 1\} \rightarrow \{-1, +1\}$, $\beta(x) = 2x - 1$, extended componentwise to bit-vectors. A word $w \in \mathbb{W}$ with binary digits $w_0, \dots, w_{31}$ is encoded as $\mathbf{b}(w) = (\beta(w_0), \dots, \beta(w_{31})) \in \{-1, +1\}^{32}$, with inverse $\beta^{-1}(y) = (y+1)/2$.

Definition 6 (State tensor). A configuration is encoded as a matrix $S \in \{-1, +1\}^{d \times K}$ whose columns are the machine’s “cells” (registers, then a memory window). Rows 0–31 hold the cell’s 32 value bits in bipolar form; rows 32–42 hold a bipolar encoding of the cell’s address/position; the remaining rows carry type and control fields, including the encoded *pc*.

The layer computes one step in three constructed stages. Opcode dispatch is *branchless*: it evaluates all 40 candidate results and selects with a one-hot mask, so the whole forward pass is a fixed sequence of tensor contractions (and thus a single fused GPU kernel).

1. **Decode and operand fetch.** An attention head reads I_{pc} ; further heads select the operand columns b, c . With keys/queries set to the position rows, the score matrix $QK^\top$ is maximised exactly at the addressed column, so a hard (or sharp soft) argmax is a *one-hot* selector $\pi \in \{0, 1\}^K$ and $S\pi$ extracts the operand.
2. **ALU.** A constructed MLP implements each opcode’s word function as a Boolean/arithmetic circuit over $\{-1, +1\}$. Bitwise ops are affine maps composed with the sign nonlinearity; addition, multiplication, shifts, comparisons and division are realised as fixed circuits on the bit rows (carry propagation, partial-product accumulation, restoring division), each producing a candidate result r_{op} .
3. **Select and write back.** With the decoded opcode as a one-hot vector e_{op} , the committed result is $r = \sum_{op} (e_{op}) r_{op}$; the *pc* update is selected the same way (fall-through, jump target, or branch target). A final *error-correction* step snaps every value to $\{-1, +1\}$ by $y \mapsto \text{sign}(y)$, restoring an exact bipolar codeword and preventing numerical drift across cycles.

Proposition 1 (One-cycle faithfulness). *Let T be the constructed layer and let enc be the state encoding. For every non-terminal configuration σ with $\sigma \longrightarrow_P \sigma'$,*

$$T(\text{enc}(\sigma)) = \text{enc}(\sigma').$$

Proof sketch. Each stage is exact by construction. The position rows make $QK^\top$ diagonally dominant at the addressed column, so the selector is exactly one-hot and operand fetch is $S\pi = \mathbf{b}(R(b))$. Each candidate circuit computes its opcode’s word function on $\{-1, +1\}$ bits: e.g. the adder reproduces $\text{mod } 2^{32}$ addition by ripple carry, and comparison circuits reproduce SLT/SLTU. The

one-hot opcode mask commits exactly the intended r_{op} and pc' . The final sign step is idempotent on $\{-1, +1\}$, so the output equals $\text{enc}(\sigma')$ bit-for-bit. Composing the constructed maps gives the claim. $\square$

Corollary 1 (Whole-program faithfulness). *Iterating T from $\text{enc}(\sigma_0)$ for k steps encodes $\sigma_0 \rightarrow_P^k \sigma_k$; hence if P halts within budget B the transformer’s decoded $\mathbf{aO}$ equals $\llbracket P \rrbracket(\sigma_0)$.*

Corollary 1 is the sense in which “running on the transformer” equals “running the program.” Independence from a CPU is structural: the value domain is $\{-1, +1\}$, the operations are tensor contractions and a sign nonlinearity, and (on GPU) the physical fault modes are entirely different from those of a scalar integer pipeline.

5 Compilation pipeline and the mini Lean runtime

5.1 From C to NISA

The front end compiles C to RV32I with an off-the-shelf cross-compiler (`riscv64-linux-gnu-gcc`, `-march=rv32im`, `-mabi=ilp32`), then translates RV32I to NISA instruction-by-instruction; a data-section pass lays out globals and materialises them with self-initialising stores, and `%hi/%lo` relocations are resolved against that layout. Two scratch registers are reserved for the translation. *This compiler is itself software and can be wrong*; two miscompilations were found and fixed during development (a global read-modify-write used as an array index; a scratch-register collision), and the differential method continues to surface codegen defects (§8). Crucially, a compiler bug here is a bug in the *auditor*: it can only cause a spurious disagreement, never a false certification, because the audit never emits a proof (§6).

5.2 Executing Lean’s emitted C

Lean compiles a `Decidable` instance to C against its runtime ABI: values are `lean_object*`, with arbitrary-precision `Nat/Int`, boxed constructors, and the `lean_nat_*/lean_int_*/lean_ctor_*` primitives. We provide a freestanding *mini runtime* implementing the fragment of this ABI that decision procedures observe:

Definition 7 (Object model). A `lean_object*` is either a *tagged scalar* — low bit 1, value $\lfloor \cdot/2 \rfloor$, used for `Nat` $< 2^{31}$ — or a heap pointer (low bit 0) into a bump arena. A heap `Nat` is a little-endian array of base- 2^{32} limbs $[n \mid \ell_0 \mid \dots \mid \ell_{n-1}]$; an `Int` is $[\text{sign} \mid \text{mag}]$ with `mag` a `Nat` object; a constructor is $[\text{tag} \mid \# \text{fields} \mid f_0 \mid \dots]$, with `List` given by `nil` = $\beta\text{-box}(0)$ and `cons` $h\ t$ = `ctor`₁ $\{h, t\}$.

Only the runtime’s own accessors inspect these private layouts, so the model is a drop-in for the ABI fragment used by decision procedures. The `Nat` layer (add, sub, mul, schoolbook long division for div/mod, square-and-multiply pow, bitwise-and, and the comparisons underlying `Nat.decEq/decLt/decLe`) is validated end-to-end on the transformer; the `Int` and constructor layers are validated against a host reference build. Because every arithmetic routine has an independent oracle (host `int`, `sympy`), the runtime is itself continuously audited by the same mechanism it enables.

Example 1 (A compiled decision, three ways). The goal `decide (a * b = c)` over `Nat` compiles to `Nat.decEq (Nat.mul a b) c : Bool`. The audit computes this Boolean (i) as Lean’s `native_decide` would, (ii) on the transformer via the mini runtime, and (iii) in a pure oracle ($a \cdot b = c$ over $\mathbb{Z}$). For $a = 2^{64} - 59$, $b = 2^{64} - 83$ (a ~ 128 -bit product) all three agree on `true`; for

the perturbed $c' = c+1$ all three agree on **false**. The false cases matter: they witness that the check is not vacuously passing.

6 The audit method

Fix a decidable predicate with intended semantics $f : X \rightarrow \mathbb{B}$. Three evaluators approximate f :

$\mathcal{E}_{\text{lean}}$ (compiled d , on CPU), $\mathcal{E}_{\text{trans}}$ (same d , via runtime $\circ$ pipeline $\circ$ transformer), $\mathcal{E}_{\text{orac}}$ (independent implementation)

native_decide accepts $P(x)$ precisely when $\mathcal{E}_{\text{lean}}(x) = \text{true}$.

Definition 8 (Audit outcome). For a finite test set $X_0 \subseteq X$, the audit reports *disagreement* if there is $x \in X_0$ with $\mathcal{E}_{\text{trans}}(x) \neq \mathcal{E}_{\text{orac}}(x)$ or with $\mathcal{E}_{\text{lean}}(x) \neq \mathcal{E}_{\text{orac}}(x)$ (when $\mathcal{E}_{\text{lean}}$ is available on x), and *agreement* otherwise.

Theorem 1 (Soundness of the falsifier). *Suppose that on input x the pipeline and transformer are faithful — i.e. $\mathcal{E}_{\text{trans}}(x) = g(x)$, where g is the mathematical function the compiled procedure denotes — and that the oracle is correct, $\mathcal{E}_{\text{orac}}(x) = f(x)$. If $\mathcal{E}_{\text{lean}}(x) = \text{true}$ while $\mathcal{E}_{\text{trans}}(x) = \text{false}$, then the compiled procedure Lean executed does not denote f at x ; consequently the **native_decide** proof of $P(x)$ rests on an unfaithful computation and $P(x)$ is not thereby established.*

Proof. By Corollary 1 and the hypothesis, $\mathcal{E}_{\text{trans}}(x) = g(x) = \text{false}$, so $g(x) \neq \text{true}$. Lean’s acceptance required the compiled evaluation to yield **true**; since $g(x) = \text{false}$, the value **native_decide** trusted differs from the denotation g of the very procedure it ran, or g differs from f . In either case the chain of **ReduceBool** relied upon — “the compiled **decide** P evaluates to **true**, therefore P ” — is instantiated with a **true** that does not reflect $f(x)$; the inference to $P(x)$ is unjustified. $\square$

Theorem 2 (Incompleteness). *Agreement on X_0 does not entail $\mathcal{E}_{\text{lean}} = f$ on X_0 , let alone on X .*

Proof. If all three evaluators share a defect δ at x (e.g. the same off-by-one), then $\mathcal{E}_{\text{lean}}(x) = \mathcal{E}_{\text{trans}}(x) = \mathcal{E}_{\text{orac}}(x) \neq f(x)$ yet the audit reports agreement. And for $x \notin X_0$ nothing is tested. Hence agreement is consistent with unsoundness. $\square$

Theorems 1–2 together say the audit is a *sound but incomplete* falsifier: it can certify the presence of a fault but never its absence. This is the formal content of the Dijkstra principle and it dictates the tactics used to enlarge coverage and independence:

Differential. Compare $\mathcal{E}_{\text{trans}}$ against $\mathcal{E}_{\text{orac}}$ over many inputs, weighted toward edges $(0, 1, 2^{31}, 2^{32}, 2^{63}$, negatives, empty structures) where fast-path discrepancies concentrate.

Metamorphic. Test relations that must hold for *all* inputs and require no separate oracle, e.g. $a \cdot b = b \cdot a$, $\text{gcd}(a, b) \cdot \text{lcm}(a, b) = a \cdot b$, or “factor then multiply back.” The identity is the oracle, closing the shared-oracle-bug gap of Theorem 2.

Bounded-exhaustive. For finite domains, enumerate $X_0 = X$; agreement then *does* entail faithfulness on X .

Redundant execution. Run the identical compiled binary on a second architecture (aarch64 vs. amd64 under emulation) and across compiler/toolchain versions; deterministic integer results must agree bit-for-bit. This is the only defence against the fault class of §7.

7 Threat model

Auditor faults are safe. A bug anywhere in LLMCompiler — front end, translator, runtime, or executor — can only make $\mathcal{E}_{\text{trans}}$ wrong. By Theorem 1 the audit emits no proof; a faulty auditor yields at worst a spurious disagreement, which is investigated and attributed. The system is fail-safe by design: it never certifies.

Shared defects. The residual blind spot of Theorem 2 is a defect common to Lean’s path and the auditor’s path. The mitigations are (i) implementation diversity — a schoolbook bignum in the runtime versus GMP-style code in Lean’s fast paths versus `sympy` in the oracle — and (ii) metamorphic checks, which do not depend on any single implementation being correct.

Silent hardware data corruption. “Mercurial cores” produce deterministic wrong results on particular data without any fault signal. Such an event inside a compiled decision procedure is a direct attack on `ofReduceBool`: `native_decide` would trust a hardware miscomputation. Redundant execution is the defence, and it is native to this method — a corruption manifests as a disagreement between two independent runs or machines, exactly what the differential and cross-architecture layers detect. (This class threatens all machine-checked proof, since the kernel runs on the same silicon; `native_decide` is merely more exposed, executing more trusted instructions without re-checking.)

Floating point. IEEE-754 $+$, $-$, $\times$, $\div$, $\sqrt{}$ are correctly rounded and hence portable and reproducible; transcendentals (`sin`, `exp`, `pow`, ...) are `libm`-defined and *not* portable. Audits of real-valued `native_decide` goals must treat only the former as cross-platform ground truth, and use interval/ball enclosure for the latter.

8 Case studies and results

- **Fast-path divergences in Lean.** Systematic differential/metamorphic sweeps of the `@[implemented_by]` surface reproduce the known pattern of reference/fast-path disagreement (e.g. an off-by-one in a `Vector` fold), the precise class Theorem 1 is designed to catch.
- **Compiler bugs found by self-audit.** Running the mini runtime through the pipeline and diffing against a host reference exposed two miscompilations (global read-modify-write indexing; scratch-register collision), both fixed, and a third still-open miscompilation of register-heavy *composed* `Int` routines. These are the method applied to its own auditor: the transformer-vs-host differential is itself a falsifier (§6).
- **Portability of the method.** The same oracle/metamorphic methodology, pointed at unrelated mature C libraries, uncovered concrete defects — for instance an infinite loop in a number-theory routine on a degenerate (zero) modulus, root-caused to a missing precondition guard in a residue search. That such a method transfers is evidence that the discipline, not the artefact under test, does the work.

9 Related work and discussion

The approach sits between three lines of work: *translation validation* and differential testing of compilers; *interval/ball arithmetic* for rigorous numerics (used here as an oracle for real-valued

goals); and the emerging study of executing programs inside neural networks and of silent hardware errors in large fleets. Its distinctive elements are (i) a weight-constructed transformer as an *exact*, independent executor with Proposition 1, and (ii) a formal account (Theorems 1–2) of what a proof-assistant audit does and does not establish. The honest scope is a semi-decision procedure for unsoundness: it strengthens confidence in proportion to coverage and independence, and it can *prove* that a particular **native_decide** is unsound, but it cannot certify soundness. For certainty one still needs kernel-level **decide** or a **Prop**-level proof; the value here is in cheaply and independently hunting for the cases where the fast, trusted path has gone wrong.

Reproducibility

The ISA, pipeline, transformer executor, and the Lean-audit toolkit (mini runtime plus a runnable three-way example) accompany this paper. The worked audit is invoked with `python -m llmcompiler.lean_audit`; the practical guide is `docs/auditing-lean-native-decide.md`.